



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Отчет по выполнению практического задания №5.1

Тема: Алгоритмы поиска в таблице при работе с данными из файла

Дисциплина: структуры и алгоритмы обработки данных

Выполнил студент
группа

Мижурин А.А.
ИКБО-50-25

Москва 2026

ЦЕЛЬ РАБОТЫ

Цель работы: Поучить практический опыт по применению алгоритмов поиска в таблицах данных.

В работе используется вариант 14. Метод поиска — поиск Фибоначчи. Запись содержит справочную информацию о городе: уникальный код города, являющийся ключом, и название города.

1 СОЗДАНИЕ БИНАРНОГО ФАЙЛА

1.1 Постановка задачи

Создать бинарный файл из записей, структура которых определяется вариантом. Для варианта 14 ключом является код города, а информационным полем — название города. Заполнить файл данными со случайными уникальными ключами. Сначала создать текстовый файл, затем преобразовать его в бинарный.

1.2 Описание подхода к решению

Запись представлена структурой `City`. Поле `code` хранит целочисленный ключ, а массив `name` — название города. Для генерации ключей применяется генератор псевдослучайных чисел, а множество `std::unordered_set` не допускает повторов. Текстовый файл содержит одну запись в строке. Во время преобразования записи считываются в исходном порядке и записываются в бинарный файл без сортировки. Каждая запись бинарного файла имеет постоянный размер `sizeof(City)`. Поэтому номер записи позволяет вычислить её смещение как

$$\text{recordNumber} \cdot \text{sizeof}(\text{City})$$

байт от начала файла.

Работа с файлами разделена между тремя функциями:

Листинг 1.1 – Прототипы функций работы с файлами

```
void generateTextFile(int count,
                     const std::filesystem::path& path);

void convertTextToBinary(
    const std::filesystem::path& textPath,
    const std::filesystem::path& binaryPath);

std::vector<City> readBinaryFile(
    const std::filesystem::path& path);
```

Листинг 1.2 – Структура записи и генерация уникальных ключей

```
struct City {
    int code;
```

```

char name[40]{};
};

void generateTextFile(int count, const std::filesystem::path& path) {
    // Генерируем случайные коды городов
    std::mt19937 rng(12345);
    std::uniform_int_distribution<int> codeDist(100000, 999999);

    // Храним уже использованные коды
    std::unordered_set<int> usedCodes;
    std::ofstream out(path);

    // Записываем заданное количество городов
    for (int i = 0; i < count; ++i) {
        int code;
        do {
            code = codeDist(rng);
        } while (!usedCodes.insert(code).second);
        out << code << " City_" << i + 1 << '\n';
    }
}

```

Предусловие: количество записей положительно, а путь доступен для создания файла.

Постусловие: текстовый файл содержит заданное число строк с уникальными кодами городов.

Листинг 1.3 – Преобразование текстового файла в бинарный

```

void convertTextToBinary(const std::filesystem::path& textPath,
                        const std::filesystem::path& binaryPath) {
    // Открываем текстовый файл и создаём массив городов
    std::ifstream in(textPath);
    std::vector<City> cities;

    // Считываем код и название каждого города
    int code;
    std::string name;
    while (in >> code >> name) {
        City city{};
        city.code = code;
        std::strncpy(city.name, name.c_str(), sizeof(city.name) - 1);
        cities.push_back(city);
    }

    // Записываем города в бинарный файл
    std::ofstream out(binaryPath, std::ios::binary);
    for (const City& city : cities) {
        out.write(reinterpret_cast<const char*>(&city), sizeof(City));
    }
}

```

Предусловие: текстовый файл доступен для чтения и содержит пары «код — название».

Постусловие: бинарный файл содержит те же записи в том же порядке.

Листинг 1.4 – Чтение записей из бинарного файла

```
std::vector<City> readBinaryFile(  
    const std::filesystem::path& path) {  
    // Открываем файл в бинарном режиме  
    std::ifstream in(path, std::ios::binary);  
    std::vector<City> cities;  
    City city;  
  
    // Считываем записи и добавляем их в массив  
    while (in.read(reinterpret_cast<char*>(&city), sizeof(City))) {  
        cities.push_back(city);  
    }  
    return cities;  
}
```

Предусловие: бинарный файл существует и содержит записи типа City.

Постусловие: функция возвращает записи файла в исходном порядке.

1.3 Результаты

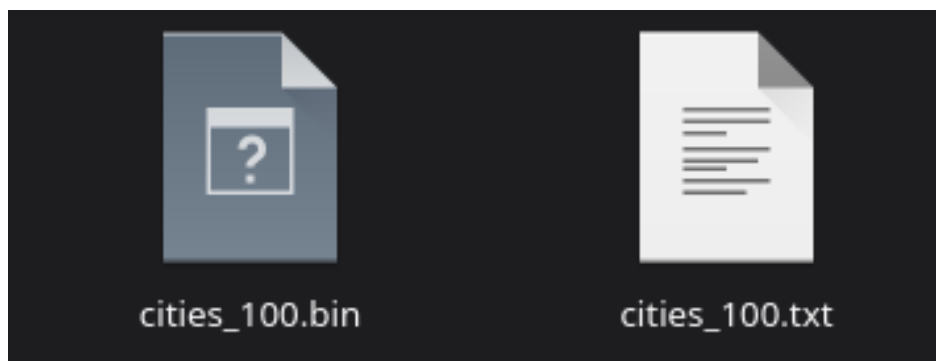


Рисунок 1.1 – Созданные текстовый и бинарный файлы для 100 записей

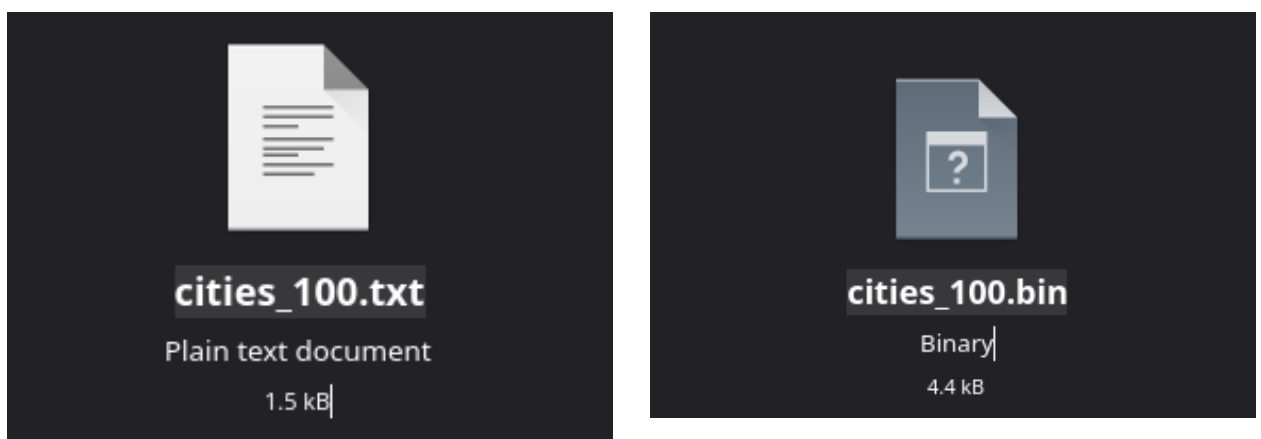


Рисунок 1.2 – Свойства файлов набора из 100 записей

Помимо набора из 100 записей программа создаёт текстовые и бинарные файлы для 1000 и 10 000 записей. Фактические размеры созданных файлов приведены в таблице 1.1.

Таблица 1.1 – Размеры файлов с исходными и бинарными данными

<i>n</i>	Текстовый файл	Размер, байт	Бинарный файл	Размер, байт
100	cities_100.txt	1492	cities_100.bin	4400
1000	cities_1000.txt	15 893	cities_1000.bin	44 000
10 000	cities_10000.txt	168 894	cities_10000.bin	440 000

Число строк в текстовых файлах равно 100, 1000 и 10 000 соответственно. Размер одной бинарной записи равен 44 байтам, поэтому размеры бинарных файлов строго пропорциональны количеству записей. Размер текстового представления дополнительно зависит от длины номера в названии City_N. Порядок записей после преобразования не изменяется.

1.4 Вывод по заданию 1

Создана структура записи варианта 14 и реализовано формирование текстового и бинарного файлов. Уникальность кодов обеспечивается при генерации, а бинарный файл сохраняет первоначальный порядок записей и допускает прямой доступ по номеру записи.

2 ЛИНЕЙНЫЙ ПОИСК

2.1 Постановка задачи

Разработать функцию поиска записи по ключу в бинарном файле методом линейного поиска. Выполнить поиск в файлах объёмом 100, 1000 и 10 000 записей, измерить время и представить результаты в таблице.

2.2 Алгоритм

1. Открыть бинарный файл для чтения.
2. Последовательно считать очередную запись.
3. Сравнить код города с искомым ключом.
4. Если ключи равны, вернуть найденную запись.
5. Если достигнут конец файла, сообщить об отсутствии записи.

Предусловие: файл содержит записи типа City, а searchKey задаёт искомый код города.

Постусловие: функция возвращает найденную запись или std::nullopt.

Листинг 2.1 – Функция линейного поиска без служебных счётчиков

```
std::optional<City> linearSearchInFile(  
    const std::filesystem::path& binaryPath,  
    int searchKey) {  
    // Открываем бинарный файл  
    std::ifstream input(binaryPath, std::ios::binary);  
    City city{};  
  
    // Последовательно проверяем каждую запись  
    while (input.read(reinterpret_cast<char*>(&city), sizeof(City))) {  
        if (city.code == searchKey) {  
            return city;  
        }  
    }  
    return std::nullopt;  
}
```

В основной программе функция вызывается после подготовки бинарного файла:

Листинг 2.2 – Вызов функции линейного поиска

```
// Ищем город по заданному коду  
const std::optional<City> foundCity =  
    linearSearchInFile(binaryPath, searchKey);
```

```

if (foundCity) {
    std::cout << foundCity->code << " " << foundCity->name << '\n';
}

```

2.3 Результаты

Таблица 2.1 – Результаты линейного поиска

n	t , мс	C	M	T
100	0,004215	200	101	301
1000	0,013673	2000	1001	3001
10 000	0,122325	20 000	10 001	30 001

Время во всех строках приведено в миллисекундах с шестью знаками после запятой. При поиске последней записи число сравнений и перемещений растёт пропорционально объёму файла, что указывает на линейную временную сложность.

2.4 Вывод по заданию 2

Линейный поиск не требует предварительной обработки файла и работает с любым порядком записей. Его недостаток — последовательный просмотр: в выбранном наихудшем случае обрабатываются все n записей.

3 ПОИСК ФИБОНАЧЧИ ПО ВСПОМОГАТЕЛЬНОЙ ТАБЛИЦЕ

3.1 Постановка задачи

Создать в оперативной памяти таблицу, содержащую ключ и ссылку на запись в файле. Реализовать поиск Фибоначчи в таблице, а затем считать найденную запись из бинарного файла методом прямого доступа. Выполнить измерения для 100, 1000 и 10 000 записей.

3.2 Описание подхода к решению

Вспомогательная таблица состоит из элементов `IndexEntry`. Поле `key` хранит код города, а `recordNumber` — номер исходной записи. Таблица сортируется по ключу, поскольку поиск Фибоначчи применим к упорядоченным данным. Сам бинарный файл не сортируется: после поиска номер записи преобразуется в байтовое смещение, и `seekg` сразу устанавливает позицию чтения.

Для организации доступа используются `std::vector` как индексируемая таблица, `std::sort` для её упорядочивания, `std::optional` для представления результата «найдено/не найдено» и `std::ifstream::seekg` для прямого доступа к записи.

Листинг 3.1 – Построение отсортированной вспомогательной таблицы

```
struct IndexEntry {
    int key;
    std::size_t recordNumber;
};

std::vector<IndexEntry> buildIndex(
    const std::filesystem::path& binaryPath) {
    // Открываем файл и создаём вспомогательную таблицу
    std::ifstream input(binaryPath, std::ios::binary);
    std::vector<IndexEntry> index;
    City city{};
    std::size_t recordNumber = 0;

    // Сохраняем ключ и номер каждой записи
    while (input.read(reinterpret_cast<char*>(&city), sizeof(City))) {
        index.push_back({city.code, recordNumber});
    }
}
```

```

    ++recordNumber;
}

// Сортируем таблицу по ключу
std::sort(index.begin(), index.end(),
    [](const IndexEntry& left, const IndexEntry& right) {
        return left.key < right.key;
    });
return index;
}

```

Предусловие: бинарный файл содержит записи одинакового размера.

Постусловие: возвращается отсортированная таблица «ключ — номер записи».

3.3 Алгоритм поиска Фибоначчи

1. Найти наименьшее число Фибоначчи, не меньшее длины таблицы.
2. Вычислить пробную позицию внутри текущей области поиска.
3. Сравнить ключ в пробной позиции с искомым ключом.
4. При меньшем ключе отбросить левую часть, при большем — правую.
5. Повторять с меньшими числами Фибоначчи до нахождения ключа или исчерпания области поиска.

Листинг 3.2 – Поиск Фибоначчи без служебных счётчиков

```

std::optional<std::size_t> fibonacciSearch(
    const std::vector<IndexEntry>& index,
    int searchKey) {
    // Создаём первые три числа Фибоначчи
    std::size_t previous = 0;
    std::size_t current = 1;
    std::size_t next = previous + current;

    // Находим число Фибоначчи не меньше размера таблицы
    while (next < index.size()) {
        previous = current;
        current = next;
        next = previous + current;
    }

    // Сужаем область поиска
    std::size_t offsetPlusOne = 0;
    while (next > 1) {
        const std::size_t position =
            std::min(offsetPlusOne + previous, index.size()) - 1;

        if (index[position].key < searchKey) {
            next = current;
            current = previous;
            previous = next - current;
        }
    }
}

```

```

        offsetPlusOne = position + 1;
    } else if (index[position].key > searchKey) {
        next = previous;
        current -= previous;
        previous = next - current;
    } else {
        return position;
    }
}

if (offsetPlusOne < index.size() &&
    index[offsetPlusOne].key == searchKey) {
    return offsetPlusOne;
}
return std::nullopt;
}

```

Предусловие: таблица отсортирована по возрастанию ключей.

Постусловие: функция возвращает позицию найденного элемента или `std::nullopt`.

Листинг 3.3 – Прямое чтение записи из бинарного файла

```

std::optional<City> readRecordAt(
    const std::filesystem::path& binaryPath,
    std::size_t recordNumber) {
    // Открываем бинарный файл
    std::ifstream input(binaryPath, std::ios::binary);
    City city{};

    // Переходим к нужной записи и считываем её
    input.seekg(static_cast<std::streamoff>(
        recordNumber * sizeof(City)));
    input.read(reinterpret_cast<char*>(&city), sizeof(City));

    if (!input) {
        return std::nullopt;
    }
    return city;
}

```

Предусловие: `recordNumber` задаёт существующую запись бинарного файла.

Постусловие: функция возвращает запись или `std::nullopt` при ошибке чтения.

Отдельные функции объединяются в программе следующим образом:

Листинг 3.4 – Поиск записи с использованием вспомогательной таблицы

```

// Строим отсортированную таблицу «ключ — номер записи»
const std::vector<IndexEntry> index = buildIndex(binaryPath);

// Ищем ключ в таблице методом Фибоначчи
const std::optional<std::size_t> position =

```

```

    fibonacciSearch(index, searchKey);

// Считываем найденную запись прямым доступом
if (position) {
    const std::optional<City> foundCity =
        readRecordAt(binaryPath, index[*position].recordNumber);
}

```

3.4 Результаты

Таблица 3.1 – Результаты поиска Фибоначчи

n	t , мс	C	M	T
100	0,003534	39	70	109
1000	0,001715	55	98	153
10 000	0,002322	66	119	185

Построение индекса выполняется до запуска таймера и в результаты поиска не включается. Измеряется поиск в готовой таблице и прямое чтение найденной записи. Количество операций растёт значительно медленнее, чем при линейном поиске; это указывает на логарифмическую временную сложность поиска в таблице.

3.5 Вывод по заданию 3

Поиск Фибоначчи требует предварительно построенной отсортированной таблицы, но не требует сортировки бинарного файла. Таблица связывает ключ с номером исходной записи, поэтому после поиска полная запись считывается одним прямым обращением к файлу.

4 АНАЛИЗ ЭФФЕКТИВНОСТИ АЛГОРИТМОВ ПОИСКА В ФАЙЛЕ

В таблице T_L и T_F обозначают число операций, а t_L и t_F — время линейного поиска и поиска Фибоначчи соответственно.

Таблица 4.1 – Сравнение эффективности алгоритмов поиска

n	t_L , мс	t_F , мс	T_L	T_F
100	0,004215	0,003534	301	109
1000	0,013673	0,001715	3001	153
10 000	0,122325	0,002322	30 001	185

При увеличении числа записей со 100 до 10 000 значение T_L выросло с 301 до 30 001, то есть примерно во столько же раз, что и объём файла. Следовательно, линейный поиск имеет линейную временную сложность. За тот же период T_F выросло только со 109 до 185, что указывает на логарифмическую временную сложность поиска Фибоначчи по готовому индексу.

Построение индекса требует линейного чтения файла. Сортировка таблицы имеет линейно-логарифмическую временную сложность. Поэтому индекс выгоден прежде всего при многократном поиске, но требует затрат памяти. Ёмкостная сложность индекса является линейной, поскольку для каждой записи файла хранится один элемент таблицы. Для одного запроса необходимо учитывать затраты на создание индекса.

ВЫВОДЫ ПО РАБОТЕ

В работе созданы бинарные файлы и реализованы два алгоритма поиска. При росте числа записей со 100 до 10 000 число операций линейного поиска увеличилось с 301 до 30 001, что подтверждает его линейную временную сложность. У поиска Фибоначчи число операций выросло со 109 до 185, что указывает на логарифмическую временную сложность по готовому индексу.

Вспомогательная таблица ускоряет повторный поиск, но требует затрат памяти. Её ёмкостная сложность является линейной, а предварительная сортировка имеет линейно-логарифмическую временную сложность.

СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. Кнут Д. Э. Искусство программирования. Том 3. Сортировка и поиск. — 2-е изд. — М.: Вильямс, 2007.
2. Страуструп Б. Язык программирования C++. Специальное издание. — М.: Бином, 2011.
3. Справочник по языку C++: потоки файлового ввода, `std::optional`, `std::filesystem` [Электронный ресурс]. — URL: <https://en.cppreference.com/> (дата обращения: 24.09.2026).